

Motion & Smoothness Upgrade

Prepared for the client · Plain-English explanation · August 2026

In one sentence: the game now stays smooth at the moments it used to stutter — mainly when a new word appears during play, and when moving between screens.

Nothing about how the game is played has changed. No rules, no screens, no artwork, no buttons moved. This work was entirely about how the app *feels* in the hand.

What was actually wrong

The game did not have a "slow animation" problem. It had a **timing collision** problem.

A phone app has one main worker that does almost everything: reading the tilt sensor, picking the next word, putting it on screen, and measuring whether it fits. Animations are supposed to be handled separately, by a second worker whose only job is sliding finished images around. That second worker is very fast and almost never gets stuck.

THE ANALOGY

Imagine one waiter running an entire restaurant. Most of the time they cope. But the moment a big table orders, everything else stops — drinks, bills, greeting people at the door. The kitchen isn't slow. The waiter is simply doing two things that landed at the same instant.

That is exactly what was happening. Every time a player tilted the phone to mark a word correct or skipped, the app had to work out what font size the new word needed so it would fit the yellow box. To do that it was **measuring the layout up to 13 times in a row** — and it was doing this *inside* the code that reads the tilt sensor, which runs 60 times a second.

So the busiest possible moment (sensor reading + choosing a word + measuring text) landed on the exact frames where the new word was supposed to animate in. The animation had nowhere to run. That is the stutter players felt.

The main fix, in plain terms

Three changes address the collision directly:

- **Stop measuring on the sensor's time.** The font-fitting now happens immediately *after* the new word is drawn, not in the middle of the sensor reading. Same result, moved out of the way.
- **Stop re-measuring what hasn't changed.** The app used to re-check the same fixed value for every single word. It now remembers it and only re-checks when the screen actually changes size or the phone is rotated.
- **Start the search from a smarter guess.** Words in a row are usually similar lengths, so the app now starts from the previous word's answer instead of starting from scratch. Most words now settle in one or

two steps instead of seven.

Screen changes: same work, better timing

Moving between screens had a similar issue. Screens like the leaderboard and the deck picker rebuild their contents from scratch. That rebuild was happening *as the fade was starting* — so the fade visibly hitched on its first frames.

The rebuild now happens a moment earlier, while the incoming screen is still hidden. The total amount of work is identical; it simply lands on a frame nobody is looking at. The fade was also shortened from 0.35s to 0.20s and given a slight upward movement and a small overshoot, so it reads as a deliberate, snappy motion rather than a slow dissolve.

THE ANALOGY

Setting the table before the guests walk in, instead of while they are sitting down. Same table, same effort — but nobody watches you do it.

Glows and effects: painted once, not every frame

Several effects in the game pulse or glow — the golden word, the premium prompts, the answer flash on the word card. These were animated in a way that forced the phone to **re-paint the entire soft glow from scratch, 60 times a second**, on some of the largest elements on screen.

They now work the way phones prefer: the glow is painted *once*, then simply faded in and out. Visually the effect is the same. The cost to the phone is a small fraction of what it was.

One background effect — a large rotating light burst — was being drawn at roughly 3800×3800 pixels on a normal phone screen, far larger than the screen itself. It is now capped at a sensible size.

Battery and background behaviour

- The floating confetti emojis on the Home screen were being created every 1.8 seconds **for the entire session** — during play, and even while the app was minimised. They now only appear when Home is actually on screen and the app is in the foreground.
- When the app is sent to the background, all decorative animation now stops. The game timer is deliberately left running, because pausing it would change how the game plays — that is a rules decision, not a smoothness one, and was not ours to make.
- Backing out of the countdown now genuinely stops it. Previously the countdown kept ticking invisibly in the background after the player had already left the screen.

Accessibility

Phones have a system setting called "Reduce Motion", used by people who get motion sickness or find heavy animation distracting. The game previously honoured that setting in **one** place while roughly 35 looping animations ignored it. It now covers the whole app: looping decoration stops, and the remaining transitions become near-instant so the app still confirms taps and answers.

A bug found and fixed along the way

Word size after rotating to landscape

While making the "remember the measurement" change, we found it would have re-introduced a bug you had already reported: rotating the phone from portrait to landscape would keep the smaller portrait text limit, so long words would have stayed tiny instead of filling the wider box.

This was caught before release and fixed — rotating now correctly clears the remembered value. Worth flagging because it is precisely the symptom you saw previously.

What we deliberately did *not* do

We did not add a physics animation engine

A common suggestion for "premium feel" is a spring-physics animation library. We advise against it here, and did not use one. Those libraries run on the *same* overloaded worker that was causing the stutter in the first place — so on a mid-range Android phone it would have made the problem worse, not better. The current approach hands animation to the phone's dedicated animation hardware, which keeps running smoothly even when the main worker is busy.

The "bouncy, snappy" character you asked for is achieved through the shape of the motion curves instead, which costs nothing.

We did not touch the tilt controls

The tilt detection code is treated as locked. It was confirmed working on a real device and has a history of breaking when altered for unrelated reasons. It was not modified, and we verified this mechanically: the tilt code was fingerprinted across all six places the game ships (website, web upload package, this new package, and the Android and iOS app bundles) and all six are **identical, character for character**.

We skipped one planned optimisation on purpose

One further technique would have let the phone skip hidden screens entirely. On the mid-range Android browsers this game targets, it cannot be delayed until after the fade finishes — outgoing screens would vanish instantly instead of fading out. The trade was not worth it, so it was left out.

Before and after, at a glance

What	Before	After
Layout measurements per word shown	Up to 13	1–2 typical
Those measurements run inside the tilt sensor loop	Yes	No
Glow effects repainted every frame	3	0
Screen change duration	350 ms fade only	200 ms, fade + movement
Heavy rebuild during the fade	Yes	Moved earlier
Oversized rotating background layer	~3800 px	Capped at 1100 px
Imprecise "animate everything" rules	6	0
Home confetti created while off-screen / minimised	Yes, all session	No
Animation while app is backgrounded	Keeps running	Stops
"Reduce Motion" accessibility coverage	1 rule	~25 rules
Countdown stops when player backs out	Kept ticking	Stops
Tilt control code	Unchanged — verified identical on all 6 builds	

What a player will actually notice

During a round

Words appear cleanly on tilt instead of catching. The new word pops in rather than sliding sideways, so each answer reads as its own beat.

Between screens

Screen changes feel quicker and more deliberate — a short push with a slight bounce, instead of a slow cross-fade that hitches at the start.

On older phones

This is where the difference should be largest. The work removed was heaviest exactly on the mid-range Android hardware most party phones are.

Battery

The app stops doing decorative work when it is not being looked at, which it previously kept doing for the whole session.

How this was checked

The new work is packaged **separately** from the current live version, so the two can be compared side by side before anything is published. The existing package was not modified in any way.

Automated checks, all passing:

Code syntax	0 errors
Tilt behaviour	43 of 43 passed
Word sizing	43 of 43 passed
Mode selection screen	17 of 17 passed
Tilt code identical	6 of 6 builds match

One thing still outstanding — please read

The figures above come from analysing the code, not from filming a phone. The claim that the game now holds a steady frame rate has **not yet been measured on a real device**.

To confirm it, we need a short recording session on a mid-range Android phone and an iPhone: entering a game from the Home screen, and roughly ten word swaps during play, before and after. That is the last step before this can be called proven rather than expected.